

# Spring Boot 가이드 1&2

구조 (정적 연결)와 동작 (요청 흐름) 함께 읽기

## 가이드 1. 파일 추적법 (정적 구조 읽기)

대상 파일 : loginauth-springboot/src/main/java/loginauth/web/AuthController.java

스프링부트 (SpringBoot) 디렉토리 및 소스 파일을 읽는 흐름 → 아무 파일이나 열었을 때 적용하는 3단계 방법입니다.

### 읽는 순서 3 단계

1. 클래스 맨 위, 생성자 바로 위에 있는 필드 목록부터 봅니다. (private final AuthService authService; 같은 줄) → "이 클래스가 누구랑 협업하는지" 명단입니다.

*이건 원래 하나였던 클래스 파일 내 메소드들을 역할과 책임별로 분리하면서 발생한 불가피한 조치입니다. 즉 책임을 나누는(Single Responsibility) 순간, "나뉜 조각들을 다시 연결해야 하는 문제"가 필연적으로 따라옵니다. DI(의존성 주입)는 그 재연결 문제를 푸는 장치가 됩니다. 이 자리를 실제로 채워주는 건 서버가 시작될 때 Spring 컨테이너입니다. 개발자는 "이런 동료가 필요하다"는 선언만 해주면 됩니다.*

2. 그 타입 이름을 파일 최상단 import 문에서 찾습니다. → 그 클래스가 몇 번째 폴더(패키지)에 있는지 나옵니다.

3. 그 경로로 직접 이동해서 파일을 엽니다. 그 파일도 또 자기만의 필드 목록이 있을 테니, 1번부터 반복합니다.

### 예시로 확인하기 : private final AuthService authService;

```
private final AuthService authService;  
           ↑ 타입      ↑ 변수명
```

- 변수명(authService): AuthController 안에서만 쓰이는 "이름표"입니다. 만든 사람이 마음

대로 지은 것이고, AuthService 클래스와는 아무 상관 없습니다.

- 타입(AuthService): "이 변수에는 AuthService 클래스로 찍어낸 객체만 담을 수 있다"는 규칙입니다. 이 타입 이름 덕분에 Spring이 "어떤 값을 넣어야 하는지"를 알 수 있습니다.

즉, 필드 선언은 "연결을 미리 준비해두는 것"이고, 함수(메서드) 안에서 .메서드()를 호출하는 건 "그 준비된 상대방에게 실제로 일을 시키는 것"입니다. 두 개는 다른 단계입니다.

## 단계 구분

- 준비 단계 (필드 선언 + 생성자) : "나는 AuthService라는 동료가 필요해" 하고 미리 자리를 마련해두는 것. 아직 아무 일도 안 시켰습니다.
- 실행 단계 (메서드 body 안에서 호출) : 실제로 그 동료에게 "이 일 좀 해줘"라고 시키는 것에 해당합니다.

## signup() 코드로 보기

```
public void signup(@Valid LoginRequest request, HttpServletResponse response) throws
IOException {

    authService.signup(        // 1) 미리 준비된 동료(authService)에게 일을 시킴
        request.username(),    // 2) request라는 매개변수 객체에게 "이름 좀 줘" 시킴
        request.password()     // 3) 마찬가지로 "비밀번호 좀 줘" 시킴
    );

    response.sendRedirect("/login"); // 4) response 객체에게 "리다이렉트 해줘" 시킴
}
```

## dto 폴더는 무엇인가?

하위에 있는 dto 폴더는 "이 API가 주고받는 데이터의 모양(형식)만 정의해두는 전용 클래스"들을 모아둔 곳입니다. DTO는 Data Transfer Object(데이터 전달 객체)의 줄임말입니다.

이유는 "실제 업무 데이터(도메인 모델)"와 "API가 주고받는 데이터"를 일부러 구분하기 위해서입니다.

- User(도메인) 객체는 id, username, 암호화된 password 등 내부 로직에 필요한 모든 걸 다

갖고 있습니다.

- 하지만 로그인 폼에서 넘어오는 데이터는 딱 username, password 두 개뿐이면 됩니다. id 는 사용자가 입력하는 것도 아니고, 알 필요도 없습니다.
- 사용자 입/출력으로 처리되는 데이터는 DTO로 분리해두면 이 둘이 서로 영향받지 않고 독립적으로 바뀔 수 있습니다.

*응답도 마찬가지로 원리입니다. check()가 반환하는 AuthCheckResponse도 User를 그대로 안 쓰고 따로 만든 응답 전용 DTO입니다.*

## 응답은 왜 항상 존재하는가?

응답을 직접 설계하지 않아도 Spring Boot(더 정확히는 HTTP 자체의 규칙)가 항상 "기본값"으로 뭔가 응답을 만들어서 보냅니다. 다만 그 기본값이 원하는 응답이 아닐 수 있다는 게 문제입니다.

HTTP는 요청을 보내면 반드시 상태 코드(200, 302, 404, 500 등)가 포함된 응답이 와야 하는 프로토콜입니다. "응답 안 함"이라는 선택지 자체가 없습니다. 그래서 개발자가 응답을 세세히 설계하지 않으면, Spring Boot가 상황에 맞는 기본 동작으로 대신 채워 넣습니다.

## REST API 판단 기준

Spring Boot가 자동으로 처리해주는 것과 무관하게, 아래 최소 기준에서 모두 "Yes"일 때 REST로 판단합니다.

- 1. 요청/응답이 구조화된 데이터(JSON)로 오가는가? (양식 데이터나 HTML이 아니라)
- 2. 응답이 데이터인가, 화면 이동 지시(redirect)인가?
- 3. 서버가 상태(세션)를 안 갖고 매 요청을 독립적으로 처리하는가?

*signup()은 이 기준에 하나도 안 맞습니다(양식 데이터, redirect 응답). 같은 컨트롤러 안의 check()와 비교해보면 그 차이가 명확해집니다.*

## 결론. 이 프로젝트에서 REST API 는 어디인가?

AuthController.java에서는 check() 메서드(GET /api/auth/check)만 위 3가지 기준을 모두 충족하는 REST API입니다. signup(), login()은 폼 데이터를 받고 redirect로 응답하는 전통적 웹 방식이며, PageController의 모든 메서드도 마찬가지입니다. 즉 이 프로젝트는 한 컨트롤러 안에 전통적 폼 처리와 REST API가 섞여 있는 하이브리드 구조입니다.

---

## 가이드 2. 동적 흐름"(요청 하나가 시간 순서대로 어떤 파일을 차례로 통과하는가?)

loginauth-springboot/src/main/java/loginauth/security/JwtAuthenticationFilter.java  
loginauth-springboot/src/main/java/loginauth/web/ApiExceptionHandler.java  
loginauth-springboot/src/main/java/loginauth/security/JwtProperties.java

가이드 1이 "파일들이 정적으로 어떻게 연결되어 있는가"였다면, 이 가이드는 "요청 하나가 시간 순서대로 어떤 파일을 차례로 통과하는가"입니다.

화살표 표기 규칙: ⇨ 는 정상적으로 요청을 보내고 응답을 돌려받는 왕복 관계, → 는 예외가 발생해서 원래 호출자에게 돌아가지 못하고 계속 위로 튕겨 올라가는 편도 흐름을 뜻합니다.

### Case A. 로그인 성공 후 상태 확인 (GET /api/auth/check, 쿠키 있음)

\* 아래 번호는 "응답이 끝나는 순서"가 아니라 "요청이 시작되는 순서"를 따릅니다.

#### 1. [JwtAuthenticationFilter.java] doFilterInternal() ⇨ [CookieService.java] readAccessToken()

요청 : 로그인 토큰이 있는지 확인하려고 쿠키를 읽어달라고 요청

응답 : 쿠키 목록에서 accessToken을 찾음 → 값이 담긴 Optional 반환

#### 2. [JwtAuthenticationFilter.java] doFilterInternal() ⇨ [JwtProvider.java] verify()

요청 : 받은 토큰 문자열이 진짜 우리 서버가 발급한 유효한 토큰인지 검증을 요청

응답 : (2번 요청 후, 3번이 요청되며, 내부적으로 3번을 거쳐) 검증된 DecodedJWT 객체를 돌려

받음 (응답은 3번이 먼저 끝남)

3. [JwtProvider.java] verify() ⇔ [com.auth0.jwt 외부 라이브러리] verifier.verify()

요청 : 자신이 미리 만들어둔 verifier에게 실제 검증을 위임

응답 : 서명·만료시간·issuer를 실제로 대조 확인 → 문제 없으면 DecodedJWT 반환

4. [JwtProvider.java] ⇔ [JwtAuthenticationFilter.java] (반환값 전달)

요청 : (3번의 결과를 그대로 자신의 호출자에게 돌려줌)

응답 : 검증된 DecodedJWT 객체가 호출했던 필터로 정상 반환됨

5. [JwtAuthenticationFilter.java] jwt.getClaim("username").asString() ⇔ [Spring Security 프레임워크 클래스] new UsernamePasswordAuthenticationToken()

요청 : 검증된 토큰 안에서 username 값을 꺼내(출처: JwtProvider.createAccessToken())

의 .withClaim("username", user.username()), 그 값으로 인증 객체 생성을 요청

응답 : 꺼낸 username으로 Spring Security가 이해하는 인증 객체가 새로 만들어져 반환됨

6. [JwtAuthenticationFilter.java] ⇔ [Spring Security 프레임워크 내부] SecurityContextHolder.setAuthentication()

요청 : 방금 만든 인증 객체를 "이 요청은 인증된 사용자다"라고 등록 요청

응답 : 요청 전체에서 공유되는 저장소에 그 인증 정보가 실제로 저장 완료됨

7. [JwtAuthenticationFilter.java] doFilterInternal() ⇔ [Spring 프레임워크 내부] filterChain.doFilter()

요청 : 자신이 할 일(토큰 검증, 인증 등록)을 다 마쳤으니 다음 단계 처리를 위임

응답 : 필터 체인의 나머지가 전부 처리를 마치고 정상적으로 돌아옴

8. [Spring 프레임워크 내부, SecurityConfig.java 참고] ⇔ [Spring 프레임워크 내부] DispatcherServlet

요청 : 인가 필터가 "/api/auth/check"가 authenticated() 목록인지 확인 요청

응답 : 6번에서 이미 인증 정보가 등록되어 있으므로 "인증됨"으로 판단, 통과시켜 넘김

9. [Spring 프레임워크 내부] DispatcherServlet ⇔ [AuthController.java] check()

요청 : 매핑된 메서드 실행을 요청. 이때 Authentication 매개변수도 함께 채워서 전달

응답 : check()가 실행되며, 이 매개변수는 Spring Security 프레임워크 내부가 SecurityContextHolder에서 자동으로 꺼내 채워 넣어준 것 (6번 결과)

#### 10. [AuthController.java] check() ⇨ [Spring Security 프레임워크 내부] authentication.getName()

요청 : 전달받은 인증 객체에서 사용자 이름을 꺼내달라고 요청

응답 : 3~6번 과정에서 저장해둔 username 값을 그대로 반환

#### 11. [AuthController.java] check() ⇨ [AuthCheckResponse.java] new AuthCheckResponse()

요청 : 꺼낸 username과 로그인 상태 정보를 담아 응답 객체를 만들려고 생성자 호출

응답 : 전달받은 값들로 객체가 만들어져 반환됨

#### 12. [AuthController.java] ⇨ [Spring 프레임워크 내부] (JSON 직렬화)

\*직렬화(Serialization)란, 메모리 안에 있는 자바 객체를 네트워크로 전송 가능한 "JSON 문자열 (텍스트)" 형태로 바꾸는 작업 (**HTTP** 는 객체를 그대로 전송할 방법이 없고, 오직 **텍스트(문자열)나 바이트**만 주고받을 수 있음)

요청 : 완성된 AuthCheckResponse 객체를 그대로 반환하며 처리를 위임

응답 : @RestController가 이 객체를 JSON 문자열로 변환, 최종 HTTP 응답으로 전송

*포인트 : 필터는 컨트롤러보다 항상 먼저 실행되고, "인증 여부"라는 상태를 SecurityContextHolder에 미리 남겨두면 이후 단계가 그걸 그대로 꺼내 쓴다는 것. Authentication 매개변수는 "마법처럼 채워지는 것"이 아니라 필터가 미리 채워놓고 간 것을 컨트롤러가 나중에 읽는 것뿐입니다. JwtAuthenticationFilter (1, 5~7번) → JwtProvider (2~4번)가 등장하고, ApiExceptionHandler는 이 시나리오엔 등장하지 않습니다(예외가 안 났으니까). 그리고 12단계 전부 ㄴ(왕복)로만 이어진다는 점이 Case B와의 가장 큰 대비입니다.*

#### Case B. 로그인 실패 (POST /api/auth/login, 비밀번호 틀림)

**1. [JwtAuthenticationFilter.java] doFilterInternal() ⇄ [CookieService.java] readAccessToken()**

**요청 :** 요청에 유효한 로그인 토큰이 있는지 확인하려고 쿠키를 읽어달라고 요청

**응답 :** 요청 쿠키 목록을 뒤져봤지만 accessToken이 없음 → 빈 Optional 반환

**2. [JwtAuthenticationFilter.java] doFilterInternal() ⇄ [Spring 프레임워크 내부] filterChain.doFilter()**

**요청 :** 받은 값이 비어 있으므로(ifPresent 스킵) 자신이 할 일은 없다고 판단, 다음 단계 처리를 위임

**응답 :** 필터 체인의 나머지(다음 필터, DispatcherServlet 등)가 전부 처리를 마치고 정상적으로 돌아옴

**3. [Spring 프레임워크 내부, SecurityConfig.java 참고] ⇄ [Spring 프레임워크 내부] DispatcherServlet**

**요청 :** 필터체인에 포함된 인가(Authorization) 필터가 SecurityConfig에 적힌 규칙을 확인 요청

**응답 :** "/api/auth/login"이 permitAll 목록에 있다고 판단, 통과시켜 넘김

**4. [Spring 프레임워크 내부] DispatcherServlet ⇄ [AuthController.java] login()**

**요청 :** 요청 URL과 HTTP 메서드로 매핑 테이블을 조회해 해당 메서드 실행을 요청

**응답 :** 일치하는 login() 메서드가 실행되기 시작함

**5. [AuthController.java] login() ⇄ [AuthService.java] authenticate()**

**요청 :** request.username(), request.password()를 그대로 넘기며 인증을 위임

**응답 :** 정상적인 반환값을 들고 돌아오지 못함 — 이 지점부터 흐름이 예외로 갈라짐(6번)

**6. [AuthService.java] authenticate() → [InvalidCredentialsException.java] 생성 - 예외 발생**

DB 조회 결과와 비교했더니 비밀번호가 일치하지 않음을 확인. throw new

InvalidCredentialsException() 실행 — 정상적인 값 반환이 아니라 예외 객체가 만들어져 즉시 던져짐.

**7. [AuthService.java] → [AuthController.java] login() - 예외 전파**

authenticate()가 정상 반환하지 못하고, 던진 예외가 호출자였던 login()으로 그대로 튀어 오름.

## 8. [AuthController.java] → [Spring MVC 프레임워크 내부] - 예외 전파

login()도 예외를 처리 못 하고 그대로 흘러보냄. Spring MVC가 처리되지 않은 예외를 감지하고 담당자를 찾기 시작.

## 9. [Spring MVC 프레임워크 내부] → [ApiExceptionHandler.java] invalid() -예외 전파 (처리 담당자 탐색)

던져진 예외의 타입(InvalidCredentialsException)과 일치하는 @ExceptionHandler를 검색해 찾아냄.

## 10. [ApiExceptionHandler.java] invalid() ⇔ [Spring 프레임워크 내부] (응답 직렬화)

**요청** : 예외를 넘겨받아 어떤 응답을 만들지 처리를 요청받음

**응답** : ResponseEntity.status(401).body(Map.of("code","INVALID\_CREDENTIALS", ...))를 반환 → 이 값이 최종 HTTP 응답(401)으로 브라우저에 전송됨

*짚을 포인트: 1~5번, 10번은 ⇔(왕복), 6~9번만 →(편도)입니다. 예외가 "발생"하는 순간(6번)부터 "처리 담당자에게 도착"하는 순간(9번)까지는 원래 호출자에게 정상적으로 돌아가지 않고 계속 위로만 튕겨 올라간다는 걸 화살표 방향 자체가 보여줍니다. 신규 파일 중에서는 JwtAuthenticationFilter(1~2번) → ApiExceptionHandler(9~10번)만 등장하고, JwtProvider는 이 시나리오엔 나오지 않습니다.*

## 두 사례를 합쳐서 보면

화살표 방향 하나만 봐도 "지금 정상 흐름인지, 예외 흐름인지"를 바로 구분할 수 있습니다. ⇔가 쭉 이어지다가 →로 바뀌는 구간이 있다면 그건 예외가 발생했다는 신호이고, 다시 ⇔로 돌아오면 어딘가(ApiExceptionHandler 같은)에서 그 예외를 붙잡아 처리했다는 뜻입니다.

같은 JwtAuthenticationFilter.java 파일 하나가 쿠키 유무에 따라 완전히 다른 결과를 만들어내고, ApiExceptionHandler와 JwtProvider는 서로 겹치지 않고 각자 다른 시나리오에서만 등장한다는 대비까지 더해지면, "이 파일이 정확히 언제, 왜 필요한가"가 이 가이드안에서 완결됩니다.